Project Synapse — Context-Aware Remediation Playbook

Role: Senior Security Automation Engineer

Objective:

This workflow was designed to simulate how suspicious API activity can be analyzed and automatically remediated within a security operations environment.

The remediation workflow simulates a real-world security incident in which a leaked API key is actively being used by multiple applications.

The playbook performs:

- Active session monitoring
- Context-aware threat analysis
- Malicious session containment
- Risk-based API key rotation
- Automated rollback handling
- Safety throttling and escalation

Scenario Overview

An API key belonging to a Global Service Account was exposed publicly in a GitHub repository.

The challenge is that the key is currently being used by over 40 production applications. Immediate revocation could result in major outages and operational failure.

The objective is therefore to:

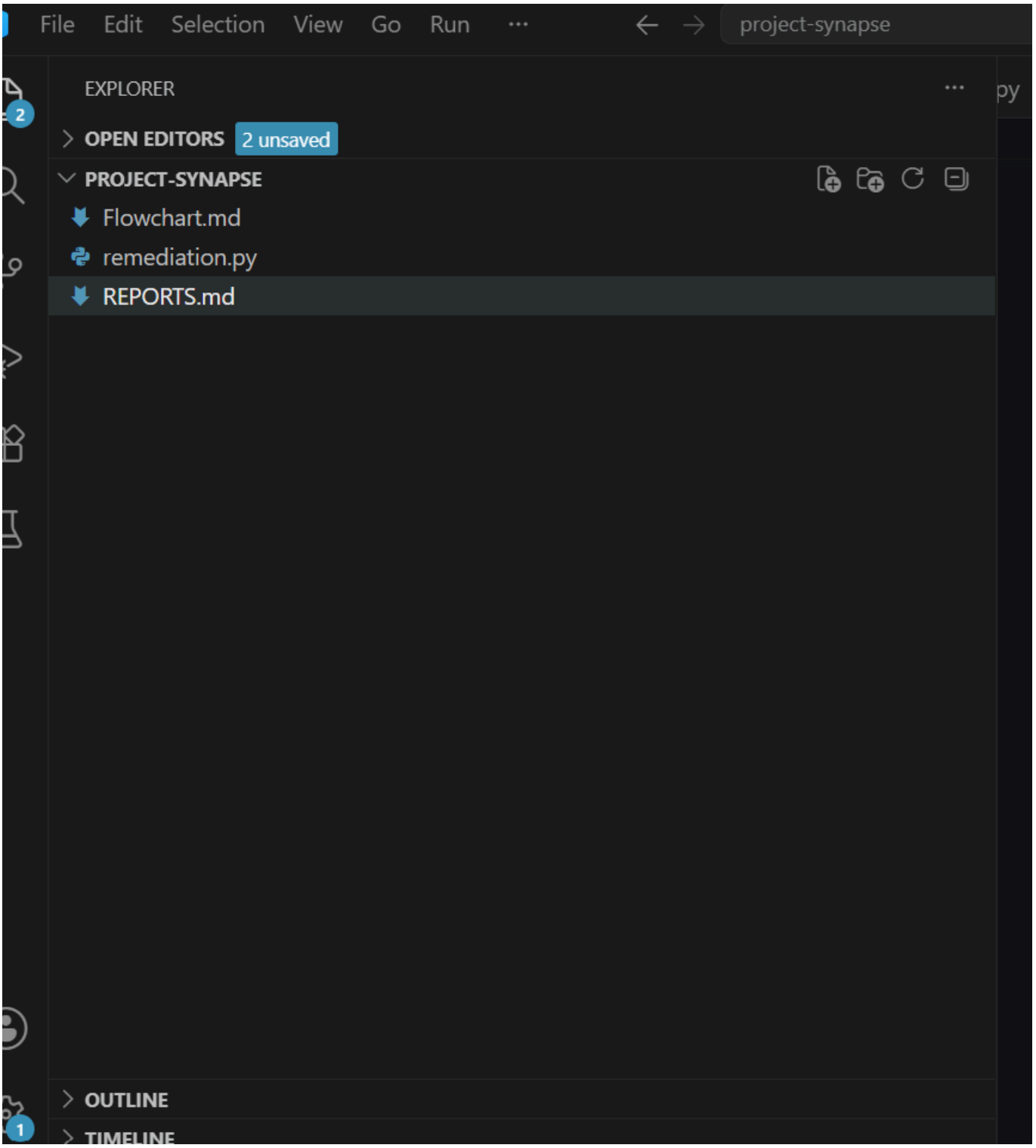
- Detect malicious usage in real-time
- Distinguish legitimate traffic from suspicious traffic
- Perform safe and phased remediation
- Reduce operational risk during key rotation

Step 1 - Environment Setup

A project workspace was created in Visual Studio Code using Python. The following project structure was created:

Project-Synapse

- remediation.py
- flowchart.md
- reports.md



Step 2 - Simulating the Mock APIs

Since mock API specifications were provided instead of real APIs, simulated API classes were implemented inside remediation.py.

The following APIs were created:

- IdentityAPI
- AssetInventoryAPI
- NotificationAPI These simulated APIs reproduce the behavior required.

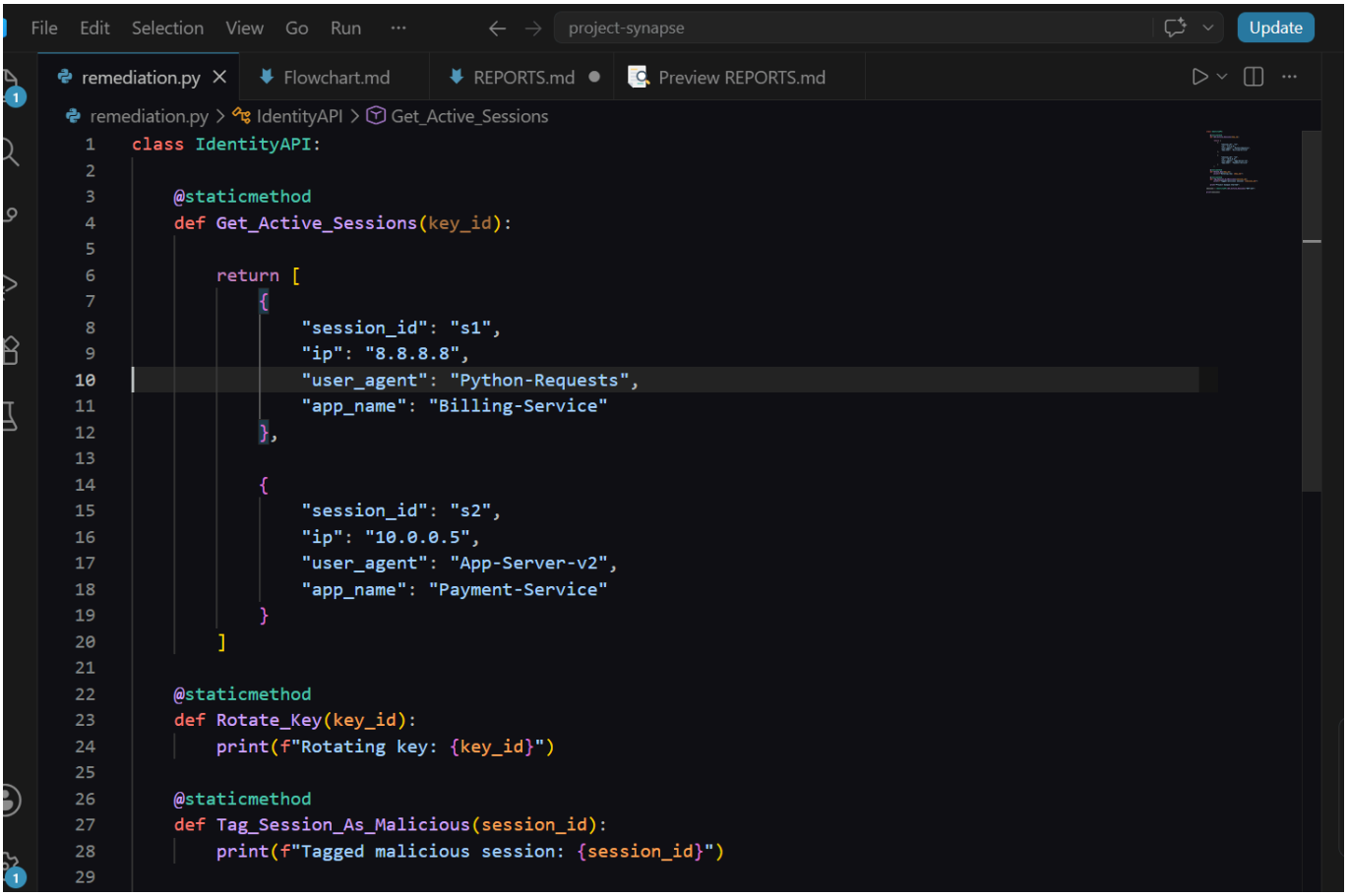
Step 3 - IdentityAPI Implementation

The IdentityAPI simulates:

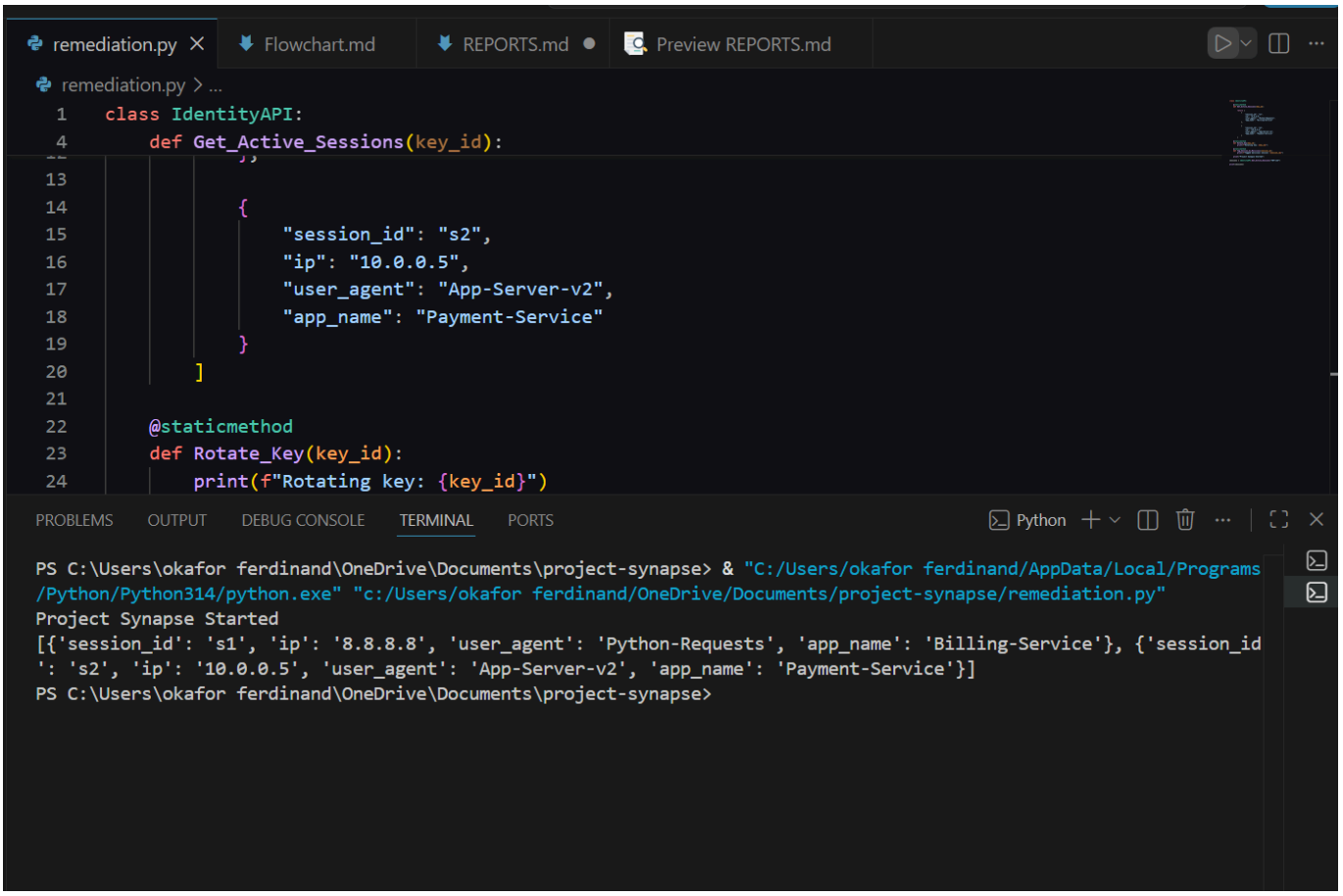
- Retrieving active sessions
- Rotating API keys
- Tagging malicious sessions

Note: Sample IP addresses, session IDs, user agents, and application names were created to simulate real-world activity within the testing environment.

screenshot 1



screenshot 2

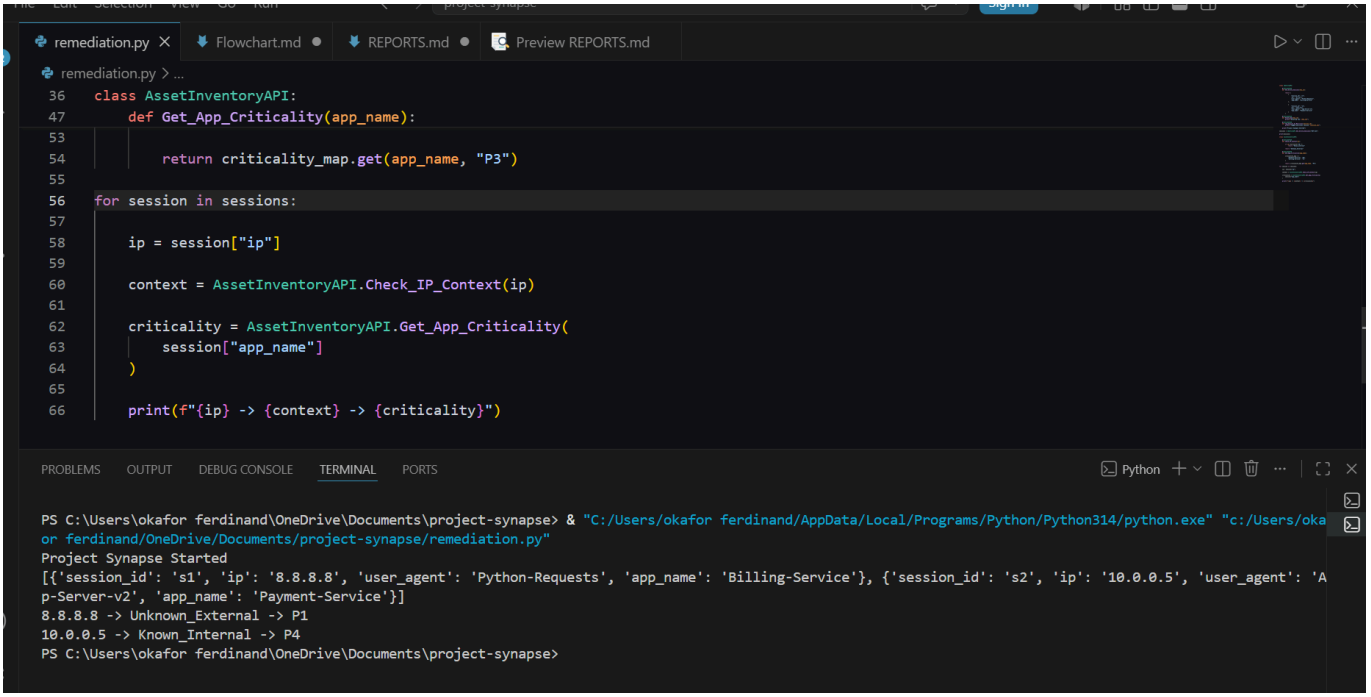


Step 4 - AssetInventoryAPI Implementation

The AssetInventoryAPI was implemented to classify IP addresses and the criticality.

The API returns:

- Known_Internal
- Unknown_External
- Application criticality



Step 5 - Automated Remediation Logic

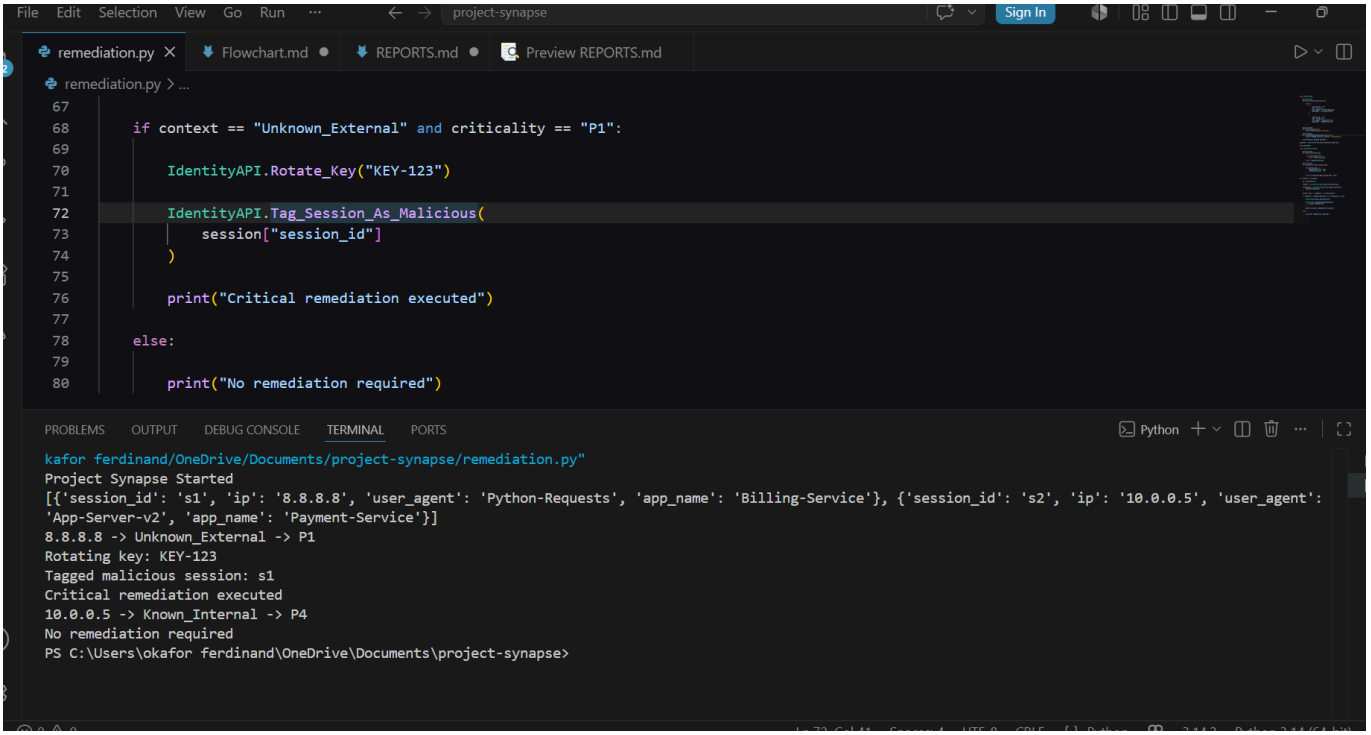
The remediation workflow was extended to automatically respond to suspicious sessions based on IP context and application criticality.

The logic checks whether:

- the IP address is classified as Unknown_External
- and the application criticality is P1

If both conditions are met, the workflow automatically:

- rotates the compromised API key
- tags the session as malicious
- executes remediation actions



Step 6 - Remediation Workflow Flowchart

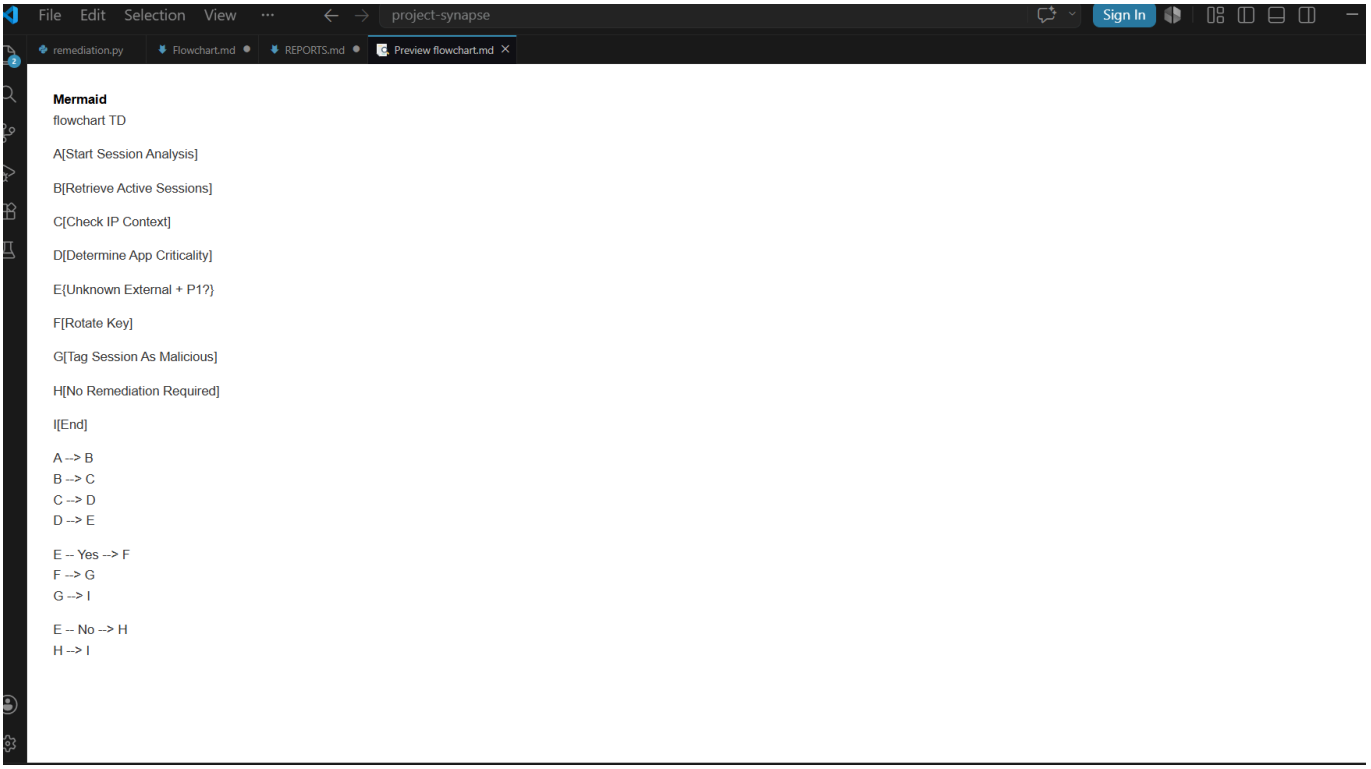
A Mermaid flowchart was created to visually represent the remediation workflow implemented in the project.

The flowchart illustrates the complete security decision-making process from session retrieval to automated remediation execution.

The workflow includes:

- retrieving active sessions from the IdentityAPI
- analyzing IP address context
- determining application criticality levels
- evaluating remediation conditions
- executing automated response actions
- The flowchart provides a simplified visual representation of how suspicious sessions are detected and handled automatically within the remediation engine.

The diagram was implemented using Mermaid syntax inside the flowchart.md file and previewed directly in Visual Studio Code.



Conclusion

The remediation workflow successfully demonstrated how suspicious API activity can be analyzed and automatically remediated using contextual security checks.

The workflow was able to:

- retrieve active sessions
- classify IP reputation
- determine application criticality
- identify high-risk activity
- execute automated remediation actions
- visualize the remediation process using Mermaid flowcharts **The implementation reflects a simplified security orchestration and automated response process commonly used in modern enterprise security environments.**